

Formules de référence — Fonction zêta de Riemann

Version enrichie — Projet Riemann_Lab · hprzeta Mise à jour : 2 juin 2026 · Auteur : hprzeta

- leçons Vérif A/B v4.1 : erreur de position (perturbation 1^{er} ordre) et distinction comptage/position
 - leçon v4.2 : finition Newton (ordre 2, dérivée analytique, critère dps vs LMFDB absolu)
 - leçon v4.2 mesurée : Newton réfuté (Z' coûteux), goulot = mp-math.siegelz, pas l'algorithme
 - §17 modèle de coût complet (mur de latence) : $t_{\text{appel}} \propto N \text{ dps}^2$, $T_{\text{total}} \approx n n_{\text{itér}} t_{\text{appel}} / W$, leviers et régimes
-

1. Définitions fondamentales

1.1 Série de Dirichlet ($\text{Re}(s) > 1$)

$$\zeta(s) = \sum_{n=1}^{\infty} \frac{1}{n^s} = 1 + \frac{1}{2^s} + \frac{1}{3^s} + \dots$$

Convergente absolument pour $\text{Re}(s) > 1$.

1.2 Produit d'Euler ($\text{Re}(s) > 1$)

$$\zeta(s) = \prod_{p \text{ premier}} \frac{1}{1 - p^{-s}}$$

Ce produit encode la distribution des nombres premiers : chaque facteur correspond à un premier p .

1.3 Prolongement analytique

$\zeta(s)$ se prolonge en une fonction méromorphe sur $\mathbb{C} \setminus \{1\}$, avec un **pôle simple** en $s = 1$ de résidu 1.

1.4 Équation fonctionnelle (symétrie $s \leftrightarrow 1 - s$)

$$\zeta(s) = 2^s \cdot \pi^{s-1} \cdot \sin\left(\frac{\pi s}{2}\right) \cdot \Gamma(1-s) \cdot \zeta(1-s)$$

1.5 Fonction ξ de Riemann (forme symétrique)

$$\xi(s) = \frac{1}{2} s(s-1) \pi^{-s/2} \Gamma\left(\frac{s}{2}\right) \zeta(s)$$

Propriété clé : $\xi(s) = \xi(1-s)$. La fonction ξ est entière et symétrique par rapport à $\text{Re}(s) = \frac{1}{2}$.

2. Hypothèse de Riemann

Énoncé : Tous les zéros non triviaux de $\zeta(s)$ vérifient $\text{Re}(s) = \frac{1}{2}$.

- **Zéros triviaux :** $s = -2, -4, -6, \dots$ (pôles de $\sin(\pi s/2)$ masqués par Γ).
 - **Zéros non triviaux :** dans la bande critique $0 < \text{Re}(s) < 1$. On les note $\rho = \frac{1}{2} + i\gamma_n$ (si HR vraie).
-

3. Fonction Z de Hardy — détection des zéros (✓ méthode utilisée dans v3)

3.1 Définition

$$\zeta\left(\frac{1}{2} + it\right) = e^{-i\theta(t)} Z(t)$$

$Z(t)$ est **réelle** pour t réel. Sa propriété essentielle :

$$Z(t) = 0 \iff \zeta\left(\frac{1}{2} + it\right) = 0$$

Détecter les changements de signe de $Z(t)$ suffit pour localiser les zéros sur la droite critique.

Pourquoi pas $\text{Re}(\zeta)$ directement ? En v1/v2, utiliser $\text{Re}(\zeta(\frac{1}{2} + it))$ comme détecteur provoquait des **faux positifs** : la rotation de phase $e^{-i\theta(t)}$ peut annuler la partie réelle sans que $\zeta = 0$. La fonction $Z(t)$ élimine ce problème.

3.2 Formule de calcul optimisée (✓ Z_fast dans v3)

$$Z(t) = \cos \theta(t) \cdot \text{Re}\left[\zeta\left(\frac{1}{2} + it\right)\right] - \sin \theta(t) \cdot \text{Im}\left[\zeta\left(\frac{1}{2} + it\right)\right]$$

Avantage : un seul appel à `zeta()` au lieu de deux.

4. Fonction $\theta(t)$ de Riemann-Siegel

4.1 Définition exacte

$$\theta(t) = \text{Im}[\ln \Gamma(\frac{1}{4} + \frac{it}{2})] - \frac{t}{2} \ln \pi$$

compute_zeros_v2.py L171 : `mp.im(loggamma(mp.mpf("0.25") + mp.mpc(0, t)/2)) - (t/2)*mp.log(mp.pi)`

4.2 Expansion asymptotique de Stirling (✓ correction v3 — ×10 à ×50 plus rapide)

Pour $t \rightarrow \infty$ (valable pour $t \geq 20$, erreur $< 10^{-15}$) :

$$\theta(t) = \frac{t}{2} \ln \frac{t}{2\pi} - \frac{t}{2} - \frac{\pi}{8} + \frac{1}{48t} + \frac{7}{5760t^3} - \frac{31}{80640t^5} + O(t^{-7})$$

Coefficients de Bernoulli (origine des termes correctifs) :

$$\frac{B_{2k}}{2k(2k-1)t^{2k-1}} : \quad \underbrace{\frac{1}{48t}}_{B_2=1/6} + \underbrace{\frac{7}{5760t^3}}_{B_4=-1/30} - \underbrace{\frac{31}{80640t^5}}_{B_6=1/42}$$

Seuil de bascule : `theta_rapide.py` utilise l'asymptotique pour $t \geq 20$ et `mpmath` exact sinon.

5. Formule de Riemann-Siegel — Calcul vectorisé $Z(t)$ (✓ **Z_batch** dans v3)

5.1 Formule principale

$$Z(t) = 2 \sum_{n=1}^{N(t)} \frac{\cos[\theta(t) - t \ln n]}{\sqrt{n}} + R(t)$$

où $N(t) = \lfloor \sqrt{t/2\pi} \rfloor$ (nombre de termes).

5.2 Terme de reste $R(t)$ — correction RS

$$R(t) = (-1)^{N-1} \cdot \tau^{-1/2} \cdot \left[C_0(u) + \frac{C_1(u)}{\pi\tau} + \dots \right]$$

avec $\tau = \sqrt{t/2\pi}$, $u = 2(\tau - N) - 1$, et :

$$C_0(u) = \Psi(u) = \frac{\cos\left[\pi\left(\frac{u^2}{2} + \frac{3}{8}\right)\right]}{\cos(\pi u)}$$

5.3 Formulation matricielle (vectorisation GPU/CPU)

Pour un tableau de points $\{t_k\}_{k=1}^M$:

$$Z(t_k) = 2 \sum_{n=1}^{N_{\max}} \frac{\cos[\theta(t_k) - t_k \ln n]}{\sqrt{n}}$$

riemann_siegel_batch.py L291 (Z_batch) :

```
phases = thetas[:, None] - ts[:, None] * log_ns[None, :] # shape (M, N_max)
Z = 2.0 * np.dot(np.cos(phases), inv_sqn) # shape (M,)
```

Gain mesuré : $\times 7$ à $\times 15$ (CPU numpy) · $\times 8$ à $\times 12$ supplémentaires (GPU GTX 960M).

(!) **Le code ci-dessus est la version NAÏVE** (un $N_{\max}$ fixe pour tout le bloc). Elle est **fausse** dès que le bloc couvre une large plage de t — voir §5.4.

5.4 (!) Piège de la vectorisation naïve — $N(t)$ variable (bug détecté session v4.1)

$N(t) = \lfloor \sqrt{t/2\pi} \rfloor$ **dépend de t** et change à chaque point du batch. Une version vectorisée naïve qui applique un $N_{\max}$ **fixe** (calculé sur le plus grand t du bloc) à **tous** les points est **fausse** : pour un point t_k donné, les termes $n > N(t_k)$ ne font pas partie de la somme de Riemann-Siegel et **faussent le signe** de $Z(t_k)$.

Symptôme mesuré (v4.1) : la première version z_batch produisait **359 désaccords de signe** vs mpmath.siegelz → autant de zéros potentiellement ratés lors d'un run long.

Solution z_vect_correct : masque booléen ligne par ligne

$$\text{mask}[k, n] = \mathbb{1}[n \leq N(t_k)]$$

Chaque ligne k n'accumule alors que ses $N(t_k)$ termes exacts :

```
N_k = np.floor(np.sqrt(ts / (2*np.pi))).astype(int) # N(t_k) propre à chaque point
mask = ns[None, :] <= N_k[:, None] # (M, N_max) booléen
phases = thetas[:, None] - ts[:, None] * log_ns[None, :]
Z = 2.0 * np.sum(np.where(mask, np.cos(phases) * inv_sqn, 0.0), axis=1)
```

Résultat après correction : **0 désaccord** sur 4 plages testées ($t \in [14, 100], [300, 400], [3000, 3100], [9900, 10000]$).

5.5 Erreur de troncature de Riemann-Siegel

La somme tronquée à la correction $C_0 + C_1$ a une erreur résiduelle **structurelle** (c'est une limite mathématique, **pas un bug**) :

$$\text{erreur}_{\text{RS}} = O(t^{-3/2}) \quad (\text{décroît quand } t \text{ augmente})$$

Niveau de correction	Précision typique
RS sans correction	$\sim 10^{-1}$
RS + C_0	$\sim 10^{-2}$
RS + $C_0 + C_1$	$\sim 10^{-3}$
RS + 3-5 termes	$\sim 10^{-8}$
mpmath.siegelz	$\sim 10^{-12}$

Conséquence pratique (v4.1) : l'écart $|Z_{\text{vect}}(t) - \text{siegelz}(t)|$ passe de 1.4×10^{-2} ($t \approx 14$) à 1.2×10^{-4} ($t \approx 9900$). Pour la **détection**, seul le **signe** compte $\rightarrow$ cette erreur n'empêche pas de localiser les bons intervalles ; l'**affinage Illinois** (§6) corrige ensuite la position exacte à 10^{-12} .

5.6 Erreur de position d'un zéro affiné sur Z_{mpfr} (leçon Vérif B v4.1)

Point clé : l'erreur de troncature §5.5 est une erreur sur l'**amplitude** de Z . Ce qui compte pour un zéro, c'est l'erreur de **position** qui en découle. Les deux sont liées, mais ne sont **pas** du même ordre de grandeur — d'où le résultat de la Vérif B.

Si l'affinage (Illinois en C, ou findroot) cherche une racine de la fonction **tronquée** $Z_{\text{mpfr}} = Z + R$ au lieu du vrai Z , il converge vers γ_{mpfr} tel que $Z_{\text{mpfr}}(\gamma_{\text{mpfr}}) = 0$, c.-à-d. $Z(\gamma_{\text{mpfr}}) = -R(\gamma_{\text{mpfr}})$. Le **développement de Taylor au premier ordre** autour du vrai zéro γ (où $Z(\gamma) = 0$) donne :

$$0 = Z_{\text{mpfr}}(\gamma_{\text{mpfr}}) \approx Z'(\gamma)(\gamma_{\text{mpfr}} - \gamma) + R(\gamma) \implies \boxed{|\gamma_{\text{mpfr}} - \gamma| \approx \frac{|R(\gamma)|}{|Z'(\gamma)|}}$$

Lecture de la formule — l'erreur de position est l'erreur d'amplitude **divisée par la pente** du croisement. Deux régimes :

| Régime du croisement | Pente $|Z'(\gamma)|$ | Erreur de position | |---|---|---| | Croisement **raide** | grande | erreur faible ($\sim 10^{-4}$) | | Croisement **plat** (zéro proche d'un extremum, ou paire de zéros proches) | petite | erreur amplifiée (jusqu'à $\sim 10^{-2}$) |

C'est pourquoi le même reste $|R(\gamma)| \sim 10^{-3}$ produit une dispersion d'erreurs de position de 10^{-4} à 10^{-2} selon le zéro : la variabilité vient de $|Z'(\gamma)|$, pas de R .

Exemple mesuré (Vérif B, bracket [350.400, 350.440]) :

Méthode	Racine trouvée	Cible
illinois_mpfr.so (RS tronquée $C_0 + C_1$)	350.424	racine de Z_{mpfr}
mpmath.findroot(siegelz) dps=50	350.408	vrai zéro de Z
Écart	0.016	\$=

L'écart $0.016 \approx 1.6 \times 10^{-2}$ est cohérent : à $t = 350$, $N = \lfloor \sqrt{350/2\pi} \rfloor = 7$ termes, $|R(t)| \sim \tau^{-5/2} \sim 10^{-3}$, et le croisement est ici relativement plat.

Conclusion architecturale : Illinois_C **pur** sur Z_{mpfr} ne peut **pas** atteindre le critère $\text{LMFDB} < 10^{-10}$ — c'est une limite **structurelle** de la troncature, pas un bug. Pour des positions précises, il faut une **finition** sur le vrai Z (`mpmath.siegelz`) après le pré-affinage C. Voir §6.4 pour les trois architectures comparées.

6. Méthode d'affinage Illinois (✓ correction v3 — 80-90% du temps de calcul)

Illinois est une variante de la **méthode de la sécante** avec correction anti-stagnation.

6.1 Algorithme (raffinement sur $[a, b]$ avec $Z(a) \cdot Z(b) < 0$)

Initialisation : $f_a = Z(a)$, $f_b = Z(b)$.

Itération :

$$c = b - f_b \cdot \frac{b - a}{f_b - f_a}$$

- Si $f_c \cdot f_b < 0$: $a \leftarrow b$, $f_a \leftarrow f_b$
- Si $f_c \cdot f_a < 0$: **correction Illinois** : $f_a \leftarrow f_a/2$ (évite la stagnation)

Convergence : superlinéaire (ordre ≈ 1.44), garantie si Z est continue et change de signe.

6.2 Paramètres v3

Paramètre	Valeur	Justification
<code>tol_affinage</code>	1e-12	Cohérent avec 35 dps (v2 utilisait 1e-20 → impossible à 35 dps)
<code>dps_affinage</code>	35	Suffisant pour $t < 10^6$, ×3 plus rapide que 50 dps
<code>maxsteps</code>	80	Convergence garantie en < 30 itérations typiquement
<code>solver</code>	"illinois"	Via <code>mpmath.findroot(..., solver="illinois")</code>

`compute_zeros_v3.py` → `parallel_scanner.py` L258 : `findroot(Z_fast, (a,b), solver="illinois", tol=1e-12, maxsteps=80)`

6.3 Comparaison Illinois v1 / v2 / v3

Version	Fichier	Fonction détecteur	solver	tol	d
v1	<code>compute_zeros_v1.py</code> L55	<code>zeta_on_critical = Re(ζ)</code>	'newton' puis 'bisect'	1e-12	5
v2	<code>compute_zeros_v2.py</code> L197-201	<code>siegelz(t)</code> scalaire	"illinois"	1e-20	5
v3	<code>parallel_scanner.py</code> L258	<code>Z_fast(t)</code> vectorisé	"illinois"	1e-12	3

Problème v1 : $\text{Re}(\zeta)$ comme détecteur → faux positifs (voir §3.1).

Problème v2 : `tol=1e-20` irréalisable à 50 dps (≈ 50 chiffres) → timeout / dépassement `maxsteps`.

Solution v3 : `tol=1e-12` cohérent avec 35 dps (12 chiffres demandés < 35 disponibles).

6.4 Trois architectures d'affinage — arbitrage vitesse / précision (leçon Vérif B v4.1)

La Vérif B a fait apparaître une **tension fondamentale** : la racine que produit le C est rapide mais imprécise (§5.6), tandis que la racine `mpmath.siegelz` est précise mais lente. Trois architectures résolvent ce compromis différemment :

Architecture	Mécanisme de la racine finale	Vitesse
Illinois_C pur (v4.1 brut)	racine de Z_{mpfr} (RS $C_0 + C_1$) en C	~4
Callback Python/C (v5)	<code>siegelz</code> appelé <i>dans</i> la boucle C	~1
Hybride post-fork (cible v4.2)	<code>Illinois_C</code> <i>pré-affine</i> , puis <code>findroot(siegelz)</code> <i>poli en Python</i>	à n

Pourquoi l'hybride post-fork préserve le parallélisme ×4 : la boucle C ne fait **aucun** appel Python (donc aucun verrou GMP partagé entre processus). Le C rend une borne approchée γ_c ; **ensuite seulement**, et **hors** de la boucle C, Python exécute :

$$\gamma = \text{mpmath.findroot}(\text{siegelz}, \gamma_c), \quad \text{dps} = 30$$

Comme γ_c est déjà à $\sim 10^{-2}$ du vrai zéro (§5.6), `findroot` converge en très peu d'itérations (la convergence superlinéaire de la sécante part d'un excellent point initial), ce qui limite le surcoût du raffinement exact.

Distinction à ne jamais confondre : ce raffinement `siegelz` ne sert **qu'à la précision des positions** (comparaison LMFDB). Il n'est **pas** nécessaire pour *vérifier* HR jusqu'à T — voir §11.1, qui montre que la vérification repose sur le **comptage**, pas sur les positions.

6.5 Finition Newton sur le vrai Z — ordre de convergence et précision (leçon v4.2)

Mesure clé (validation hybride v4.2) : la finition `findroot(siegelz)` après pré-affinage C donne Écart_P = 0.00 (vs LMFDB) sur les trois plages $t \approx 350, 1000, 9900$ — précision **parfaite**, critère $< 10^{-10}$ largement battu. Reste à optimiser la **vitesse** du raffinement, où le bon levier est le **nombre d'évaluations** de `siegelz`, pas le dps.

6.5.1 Pourquoi Newton plutôt qu'Illinois pour la finition

La vitesse de convergence d'un solveur se mesure par son **ordre** p : si ε_n est l'erreur au pas n , alors $\varepsilon_{n+1} \approx C \varepsilon_n^p$.

Solveur	Ordre p	Évaluations / pas	Évaluations pour passer 10^{-2}
Illinois (sécante modifiée)	≈ 1.44	1	~27
Newton	2 (quadratique)	2 (Z et Z')	~6 (3 pas)

Partant du point initial fourni par le C, $\varepsilon_0 = 1.7 \times 10^{-2}$ (§5.6), Newton **double le nombre de chiffres exacts à chaque pas** :

$$\varepsilon_0 = 1.7 \times 10^{-2} \rightarrow \varepsilon_1 \approx 3 \times 10^{-4} \rightarrow \varepsilon_2 \approx 1 \times 10^{-7} \rightarrow \varepsilon_3 \approx 1 \times 10^{-14}$$

3 pas suffisent pour passer sous 10^{-10} , soit ~ 6 évaluations contre ~ 27 pour Illinois. On pourrait croire à un gain $\approx 4.5\times$. **C'est faux ici** — et la mesure l'a confirmé (§6.5.5). Le coût total est $n_{\text{évals}} \times \text{coût/éval}$, mais les deux solveurs n'évaluent pas la même chose :

- Illinois n'évalue que Z (1 appel `siegelz` par pas, ~ 296 ms à $t \approx 9000$) ;
- Newton évalue Z **et** Z' (2 appels par pas), or Z' est **plus cher** que Z (§6.5.2).

Le produit $n_{\text{évals}} \times \text{coût/éval}$ finit donc comparable. **La réduction du nombre d'itérations est réelle mais sans effet**, parce que le coût par évaluation domine et que le goulot est la vitesse intrinsèque de `mpmath.siegelz` à grand t , **pas** le nombre de pas.

Leçon épistémique : l'analyse d'ordre de convergence ci-dessus était une **prédiction a priori**, valable seulement sous l'hypothèse « coût/éval constant et Z' aussi cheap que Z ». Cette hypothèse est **fausse** (§6.5.2) ; la mesure (§6.5.5) prime sur la prédiction.

Garde-fou : Newton n'est **pas borné**. Si le croisement est plat (Z' petit, §5.6) ou sur une paire de zéros proches, l'itéré peut sortir du bracket $[\gamma_c \pm \delta]$. Prévoir un **fallback Illinois borné** dans ce cas.

6.5.2 Dérivée analytique obligatoire — annulation catastrophique

Newton requiert $Z'(t)$. Une **différence finie** centrée

$$Z'(t) \approx \frac{Z(t+h) - Z(t-h)}{2h}$$

soustrait deux nombres quasi égaux : **annulation catastrophique** qui détruit $\sim$ la moitié des chiffres significatifs. Une dérivée imprécise abaisse l'ordre effectif de Newton ($2 \rightarrow \approx 1.6$, comme la sécante) et peut faire **échouer** la convergence à 10^{-14} .

Solution : la dérivée **analytique** `mpmath.siegelz(t, derivative=1)` (pas de différence finie). Elle est calculée exactement, sans annulation, et **garantit** la cascade quadratique du §6.5.1.

(!) Mais elle n'est PAS gratuite (idée fausse corrigée par la mesure, §6.5.5). `siegelz` ne renvoie pas Z' « en bonus » d'un calcul de Z : il le **recalcule** à part. Or

$$\zeta'(s) = - \sum_{n \geq 1} \frac{\ln n}{n^s}$$

porte des **poids logarithmiques** $\ln n$ absents de $\zeta(s) = \sum n^{-s}$. Cette série converge **plus lentement**, donc l'évaluation de Z' est **plus coûteuse** que celle de Z , pas comparable. C'est précisément ce qui annule le gain d'itérations de Newton (§6.5.1).

6.5.3 Choix du dps — critère LMFDB absolu vs chiffres significatifs

Piège : le critère LMFDB est **absolu** ($|\gamma_{\text{calc}} - \gamma_{\text{LMFDB}}| < 10^{-10}$), alors que le dps contrôle un nombre de chiffres **significatifs**. Plus γ est grand, plus il faut de chiffres pour la **même** précision absolue.

Pour garantir 10^{-10} **absolu** à hauteur γ , il faut un nombre de chiffres significatifs :

$$\#\text{chiffres} \gtrsim \log_{10}(\gamma) + 10$$

À $\gamma \approx 9999$: $\log_{10}(9999) \approx 4$, donc $\#\text{chiffres} \gtrsim 14$.

dps	Chiffres dispo	Marge au-dessus de 10^{-10} absolu à $\gamma \approx 9999$	Verdict
15	15	~1 chiffre	× insuffisant (aucune marge)
20	20	~6 chiffres	(!) à valider sur le sommet
25	25	~11 chiffres	✓ sûr
30	30	~16 chiffres	✓ confortable

(!) Le test sur T=1000 ne révèle PAS ce problème : à T=1000, $\gamma < 396$ (~3 chiffres avant la virgule), donc même dps=15 paraît suffisant. Le risque n'apparaît qu'au **sommet**. → **Valider tout abaissement de dps sur un échantillon $t \approx 9900$ -10 000, jamais sur T=1000.**

6.5.4 Heuristique de coût *a priori* (!) réfutée par la mesure — voir §6.5.5)

Ce qui suit est la **prédiction** faite avant mesure. Elle s'est révélée **trop optimiste** : elle suppose 6 évaluations toutes au même coût que Z , alors que les évaluations de Z' sont plus chères (§6.5.2). Conservée ici pour la traçabilité du raisonnement.

Le coût d'une opération multi-précision croît comme dps^2 . En passant de 30 à 25 dps : $(25/30)^2 \approx 0.69$. Avec Newton (6 évaluations, ~300 ms/éval à dps=30 et $t \approx 9000$) :

$$6 \times (300 \text{ ms} \times 0.69) \approx 1.24 \text{ s/zéro} \Rightarrow \sim 0.8 \text{ z/s (1 worker)} \Rightarrow \sim 3.2 \text{ z/s} \times 4 \text{ workers}$$

soit T=10000 en ~50 min. **Prédiction non confirmée** : la mesure donne ~0.4 z/s (§6.5.5).

6.5.5 Résultat mesuré — le goulot est siegelz, pas l'algorithme (leçon décisive v4.2)

Mesure (Vérif B v3, dps=25, $t \approx 9000$) : Newton + dps=25 donne **0.4 z/s — plus lent** que Illinois + polish dps=30 (**0.5 z/s**). La prédiction du §6.5.1/§6.5.4 est **réfutée**. Précision toujours parfaite ($\text{Écart}_P = 0.00$) dans les deux cas.

Bilan de performance — toutes les options comparées ($\times 4$ workers, $t \approx 9000$) :

Approche	z/s $\times 4$	Précision (vs LMFDB)	Commentaire
Illinois_C pur (commit d9bb267)	41	$\sim 10^{-4} \times$	racines de Z_{mpfr} , pas LMFDB
Illinois + polish findroot dps=30	0.5	0.00 ✓	27 itér $\times \sim 296$ ms
Newton + dps=25, 5 pas	0.4	0.00 ✓	Z' cher $\rightarrow$ aucun gain

Cause racine confirmée : à $t \approx 9000$, un appel `siegelz` coûte ~ 296 ms (somme RS à $N = \lfloor \sqrt{t/2\pi} \rfloor \approx 37$ termes, chaque opération multi-précision en $O(\text{dps}^2)$). Newton consomme ~ 2 appels/pas (dont Z' , plus cher, §6.5.2). **Le goulot est la vitesse intrinsèque de `mpmath.siegelz` à grand t , pas le nombre d'itérations.** Réduire les itérations ne sert donc à rien : il n'y a **pas d'optimisation algorithmique possible à ce niveau** tant que l'affinage final passe par `siegelz`.

Conséquence — deux livrables, deux régimes (rappel §11.1, distinction comptage/position) :

Run	Zéros	Temps $\times 4$ estimé	Usage
T=300	138	~ 60 s (mesuré, tout en fallback <code>mpmath</code> $t < 300$)	test
T=1000	~ 396	~ 16 min	validation complète f
T=10000	$\sim 10\,142$	~ 7 h	catalogue de positi

Les **20 premières références LMFDB** ($t < 78$) tombent toutes dans la zone de fallback `mpmath` ($t < 300$) : elles sont **toujours** précises à $< 10^{-10}$, sans aucun polish.

Décision ouverte : pour vérifier HR jusqu'à $T=10000$, Illinois_C pur (41 z/s, ~ 5 min) suffit déjà (comptage Turing, §11.1). Le polish lent (~ 7 h) n'est requis **que** pour produire un **catalogue de positions** comparables à LMFDB — or ce catalogue existe déjà via v2/v3 (CSV 10 142 zéros, 50 dps). À trancher selon le livrable visé, **pas** à subir par défaut.

7. Pas de balayage adaptatif STEP (✓ correction v3)

7.1 Espacement moyen entre zéros consécutifs

La formule de Riemann-von Mangoldt donne l'espacement moyen :

$$\langle \delta_n \rangle \approx \frac{2\pi}{\ln(T/2\pi)}$$

7.2 Formule du pas de sécurité

$$\text{STEP} = \min\left(\frac{2\pi}{5 \cdot \ln(T_{\text{max}}/2\pi)}, 0.10\right)$$

Division par 5 (non par 3 comme en v2) pour sécurité contre les paires de zéros proches.

Plafond absolu à **0.10** pour $T < 500\,000$.

(!) Bug noté dans le code v3 : compute_zeros_v3.py L220 retourne `min(round(step_theorique, 3), 0.02)` alors que les commentaires L207 et L213-215 spécifient 0.10. La valeur documentée **cible** est **0.10** (intention v3). Corriger dans la Phase C ou un patch v3.1.

compute_zeros_v3.py L205-219 :

```
def step_adaptatif(T_MAX: float) -> float:
    # STEP sûr = min(espacement_moyen / 5, 0.10)
    espacement_moyen = 2 * math.pi / math.log(T_MAX / (2 * math.pi))
    step_theorique = espacement_moyen / 5.0
    return min(round(step_theorique, 3), 0.10) # plafond cible
```

Valeurs typiques :

$T_{\max}$	Espacement moyen	STEP (cible v3)
1 000	1.24	0.10 (plafonné)
10 000	0.84	0.10 (plafonné)
100 000	0.70	0.10 (plafonné)
500 000	0.60	0.10 (plafonné)

8. Formule de Riemann-von Mangoldt — Comptage $N(T)$ (✓ correction critique v3)

8.1 Formule exacte

$$N(T) = \frac{\theta(T)}{\pi} + 1 + S(T)$$

où $S(T) = \frac{1}{\pi} \arg \zeta\left(\frac{1}{2} + iT\right)$ est la **correction par variation d'argument**.

Propriété : $|S(T)| \leq C \cdot \frac{\ln T}{\ln \ln T}$, et typiquement $|S(T)| < 3$ pour $T < 10^6$.

8.2 Estimation asymptotique (Weyl — premier terme) — FORMULE CORRIGÉE

$$N(T) \approx \frac{T}{2\pi} \ln \frac{T}{2\pi e}$$

(!) Erreur classique corrigée en v3 : la formule incorrecte $\frac{T}{2\pi} \ln \frac{T}{2\pi}$ (sans le e au dénominateur) sous-estimait $N(T)$ de **~64%** pour $T = 100\,000$ (49 346 au lieu de 138 067 zéros attendus), faussant toutes les estimations de temps.

Comparaison numérique :

T	$\frac{T}{2\pi} \ln \frac{T}{2\pi e}$ (correcte)	$\frac{T}{2\pi} \ln \frac{T}{2\pi}$ (incorrecte)	Erreur
1 000	396	229	−42%
10 000	10 142	5 968	−41%
100 000	138 067	49 346	−64%

8.3 Calcul de $S(T)$ par variation continue d'argument (Backlund, 1914)

$$S(T) = \frac{1}{\pi} \int_{\sigma=+\infty}^{\sigma=1/2} \frac{\partial}{\partial \sigma} \arg \zeta(\sigma + iT) d\sigma$$

Implémentation discrète (turing_validation.py L127-158) :

```
# Chemin horizontal :  $\sigma$  de 3.0 (où  $\arg \zeta \approx 0$ ) à  $\frac{1}{2}$ 
sigmas = np.linspace(3.0, 0.5, 51) # 50 pas
variation = 0.0
arg_prev = 0.0

for sigma in sigmas[1:]:
    s = mpc(sigma, T)
    z = zeta(s)
    arg_cur = float(mp.atan2(im(z), re(z)))
    # Correction de branche – suivi continu (évite les sauts de  $\pi 2$ )
    d = arg_cur - arg_prev
    if d > math.pi: arg_cur -= 2 * math.pi
    if d < -math.pi: arg_cur += 2 * math.pi
    variation += arg_cur - arg_prev
    arg_prev = arg_cur

S_T = variation / math.pi
```

Pourquoi démarrer à $\sigma = 3$? Pour $\sigma \geq 3$, $|\zeta(\sigma + iT) - 1| < 0.2$, donc $\arg \zeta \approx 0$ — pas d'ambiguïté de branche.

9. Précision adaptative (✓ correction v3)

Stratégie à 3 niveaux selon l'opération :

Opération	Précision	Justification
θ asymptotique (float64)	15 chiffres	Numpy/Python natif — $\times 50$
Détection Z_{fast}	25 dps	Suffisant pour signe de $Z(t)$
Affinage Illinois	35 dps	Atteint $\text{tol} = 1e-12$ en 35 dps
Validation / publication	50 dps	Pour les 1000 premiers zéros

Avant correction : tout à 50 dps $\rightarrow \theta$ via `loggamma mpmath` $\rightarrow 3\times$ plus lent.

10. Espacements normalisés et conjecture de Montgomery

10.1 Espacement normalisé

$$\delta_n = (\gamma_{n+1} - \gamma_n) \cdot \frac{\ln(\gamma_n/2\pi)}{2\pi}$$

10.2 Distribution GUE (Wigner-Dyson)

La conjecture de Montgomery (1973) affirme que $\{\delta_n\}$ suit la distribution GUE des matrices aléatoires :

$$p(s) = \frac{\pi}{2} s \cdot e^{-\pi s^2/4}$$

Tracé dans v3 : histogramme de $\{\delta_n\}$ vs courbe théorique GUE.

11. Validation Turing-Backlund (✓ ajout v3)

Critère de complétude du calcul :

1. Calculer $N_{\text{exact}}(T) = \lfloor \theta(T)/\pi \rfloor + 1 + \text{round}(S(T))$
2. Compter $n_{\text{calc}} = |\{t_k \leq T\}|$
3. **Complet** $\iff n_{\text{calc}} = N_{\text{exact}}(T)$

Appliqué aux points de contrôle $T \in \{T_{10\%}, T_{25\%}, T_{50\%}, T_{75\%}, T_{\text{max}}\}$.

11.1 Comptage vs position — pourquoi Vérif A réussit alors que Vérif B échoue

Idée centrale (leçon Vérif B v4.1) : vérifier HR jusqu'à T et cataloguer les positions précises des zéros sont **deux objectifs distincts** qui ne demandent pas la même chose. La méthode de Turing valide le **nombre** de zéros, jamais leur position.

La vérification numérique de HR sur $[0, T]$ par Turing-Backlund repose sur l'égalité :

$$\underbrace{n_{\text{calc}}}_{\text{nombre de changements de signe de } Z(t) \text{ sur } [0, T]} = \underbrace{N(T)}_{\frac{\theta(T)}{\pi} + 1 + S(T) \text{ (comptage théorique)}}$$

Si cette égalité tient, alors $Z(t)$ possède exactement $N(T)$ zéros réels sur $[0, T]$; or Z a autant de zéros réels que ζ en a sur la droite critique jusqu'à hauteur T , et $N(T)$ compte **tous** les zéros non triviaux de cette bande jusqu'à T . L'égalité force donc **tous** ces zéros à être simples et sur la droite critique : **HR est vérifiée jusqu'à T** .

Conséquence directe — cette condition ne fait intervenir que le **dénombrement** des changements de signe. Déplacer chaque position raffinée de γ vers γ_{mpfr} d'un

montant $\sim 10^{-2}$ (§5.6) **ne change pas** combien il y a de changements de signe. D'où le résultat de validation v4.1, qui n'est **pas** contradictoire :

Vérif	Ce qu'elle teste	Résultat v4
A	comptage Turing + LMFDB sur les 20 premiers (fallback mpmath, $t < 78$)	✓ 138/138
B	précision des positions à grand t (Illinois_C pur)	× erreurs

En résumé : v4.1 (Illinois_C pur, 41 z/s) suffit pour **vérifier** HR jusqu'à $T = 10\,000$ via Turing ; il ne suffit **pas** pour produire un catalogue de positions $< 10^{-10}$. Ces deux livrables sont séparés.

11.2 (!) Condition de validité — la détection ne doit ni rater ni inventer un croisement

L'argument §11.1 tient **sous une hypothèse** : que le comptage des changements de signe soit **exact**, c.-à-d. que la détection ne **manque** aucun croisement et n'en **invente** aucun. Près d'un zéro où $|Z|$ est minuscule, l'erreur d'amplitude de la troncature (§5.5) pourrait **inverser un signe** et fausser le comptage — c'est exactement le mécanisme du **bug z_batch float64** (jusqu'à 359 désaccords de signe, voir §5.4), corrigé par `z_vect_correct`.

Garde-fous en place :

- détection via `z_vect_correct` (masque $n \leq N(t_k) \rightarrow 0$ désaccord sur les 4 plages testées (§5.4) ;
- pas de balayage STEP sécurisé (§7) $\rightarrow$ évite de sauter une paire de zéros proches ;
- **Turing COMPLET** sur le run = preuve *a posteriori* que le comptage est bon (aucun manquant).

Tant que Turing ressort COMPLET, l'égalité §11.1 est satisfaite et HR est vérifiée sur l'intervalle — indépendamment de la précision des positions individuelles. C'est le point de vigilance n°1 à surveiller au-delà de $T = 10\,000$.

12. 20 premiers zéros (référence LMFDB)

n	γ_n (partie imaginaire)
1	14.134725141734693
2	21.022039638771555
3	25.010857580145688
4	30.424876125859513
5	32.935061587739189
6	37.586178158825671
7	40.918719012147495
8	43.327073280914999
9	48.005150881167159
10	49.773832477672302

n	γ_n (partie imaginaire)
11	52.970321477714460
12	56.446247697063246
13	59.347044002602353
14	60.831778524609882
15	65.112544048081607
16	67.079810529494173
17	69.546401711173978
18	72.067157674481890
19	75.704690699083934
20	77.144840069680455

Seuil de validation LMFDB : $|\gamma_n^{\text{calc}} - \gamma_n^{\text{LMFDB}}| < 10^{-10}$.

13. Lien avec les nombres premiers (formule explicite de Riemann)

$$\pi(x) = \text{li}(x) - \sum_{\rho} \text{li}(x^{\rho}) + \int_x^{\infty} \frac{dt}{t(t^2 - 1) \ln t} - \ln 2$$

La somme porte sur tous les zéros non triviaux ρ (et leurs conjugués).

Conséquence de HR :

$$|\pi(x) - \text{li}(x)| = O(\sqrt{x} \ln x)$$

14. Résultats numériques — Phase 0 (compute_zeros_v3, T = 10 000)

Indicateur	Valeur
Zéros calculés	10 142
T_{max}	9 998.85
Vitesse	~3.59 z/s (batch CPU)
Score LMFDB	20/20 zéros à $< 10^{-10}$
Validation Turing	✓ COMPLET (aucun manquant)
Méthode détection	Riemann-Siegel vectorisé (Z_batch)
Méthode affinage	Illinois (35 dps, tol 10^{-12})

15. Formules appliquées dans compute_zeros_v1.py (historique)

Version v1 : première implémentation, abandon vers $t \approx 432$ (overflow GMP).

15.1 Détecteur de zéros — $\text{Re}(\zeta)$ sur la droite critique

compute_zeros_v1.py L45 :

```
# Détecteur : partie réelle de  $\zeta(\frac{1}{2} + it)$  — MÉTHODE INCORRECTE
# Cause des faux positifs (voir §3.1 pour explication)
return float(zeta(0.5 + 1j * t).real)
```

Formule sous-jacente (incorrecte comme détecteur) :

$$f(t) = \text{Re}\left[\zeta\left(\frac{1}{2} + it\right)\right]$$

15.2 Critère de validation du zéro

compute_zeros_v1.py L132 : if zeta_abs < 1e-9

$$\left|\zeta\left(\frac{1}{2} + it_0\right)\right| < 10^{-9} \implies t_0 \text{ accepté comme zéro}$$

15.3 Méthode d'affinage : Newton puis bisection

compute_zeros_v1.py L55, L65 :

```
# Tentative 1 : Newton (rapide mais peut diverger)
zero = findroot(zeta_on_critical, t_mid, solver='newton', tol=1e-12, maxsteps=50)

# Fallback : bisection (robuste, convergence linéaire)
zero = findroot(zeta_real, (t_left, t_right), solver='bisect', tol=1e-12)
```

Problème Newton : diverge si la dérivée est petite (zéro simple mais proche d'un extremum de $\text{Re}(\zeta)$).

Remplacé par : Illinois en v2/v3 (voir §6).

16. Formules appliquées dans compute_zeros_v2.py

Version v2 : Illinois introduit, siegelz scalaire, tol=1e-20 (trop strict).

16.1 Fonction θ exacte via loggamma

compute_zeros_v2.py L171 :

```
theta = (
    mp.im(loggamma(mp.mpf("0.25") + mp.mpc(0, t) / 2))
    - (t / 2) * mp.log(mp.pi)
)
```

Formule correspondante :

$$\theta(t) = \text{Im}\left[\ln \Gamma\left(\frac{1}{4} + \frac{it}{2}\right)\right] - \frac{t}{2} \ln \pi$$

16.2 Détecteur $Z(t)$ scalaire via siegelz

compute_zeros_v2.py remplace $\text{Re}(\zeta)$ par $\text{siegelz}(t)$ de mpmath :

$$Z(t) = \text{siegelz}(t) \quad (\text{mpmath} \text{ — précision } 50 \text{ dps})$$

Amélioration v2 : plus de faux positifs.

Problème v2 : appel scalaire — pas de vectorisation → lent.

16.3 Illinois en v2 avec $\text{tol}=1\text{e-}20$ irréalisable

compute_zeros_v2.py L191–201 :

```
def affiner_zero(t_gauche: float, t_droite: float, tol: float = 1e-20) -> float:
    t0 = findroot(
        lambda t: siegelz(t),
        (t_gauche, t_droite),
        solver="illinois",
        tol=tol,          # 1e-20 à 50 dps → ~1e-48 impossible
        maxsteps=100     # dépassé systématiquement
    )
```

Problème : $\text{tol}=1\text{e-}20$ exige 20 chiffres exacts dans la solution, mais à 50 dps l'illinois ne converge pas en 100 steps — timeout systématique sur les zéros rapprochés.

Solution v3 : $\text{tol}=1\text{e-}12$ à 35 dps (voir §6.2).

17. Le mur de latence du calcul — modèle de coût complet (synthèse)

Synthèse des formules établies le 2 juin 2026. Version animée : docs /animation_mur_latence.html. Rassemble en un seul endroit le coût de localisation d'un zéro précis et les leviers d'optimisation. Renvois : troncature §5.5, erreur de position §5.6, goulot mesuré §6.5.5, espacement §7.1, comptage §8 et §11.1.

17.1 Trois niveaux de coût emboîtés

Localiser **un** zéro précis n'est pas une opération atomique : c'est une chaîne d'évaluations de Z , chacune étant elle-même une somme de N termes. Trois niveaux s'emboîtent.

Niveau 1 — coût d'une évaluation $Z(t)$. La somme de Riemann-Siegel (§5.1) a N termes, et chaque opération multi-précision à dps chiffres coûte $O(\text{dps}^2)$:

$$t_{\text{appel}} \propto N \cdot \text{dps}^2, \quad N = \left\lfloor \sqrt{\frac{t}{2\pi}} \right\rfloor.$$

Conséquence directe : t_{appel} **croît avec la hauteur t** (via $N \sim \sqrt{t}$).

Niveau 2 — coût d'un zéro. L'affinage (sécante / Illinois, §6.1) est une **récur-rence** : l'itération $k+1$ a besoin du résultat de l'itération k .

$$c = b - Z_b \cdot \frac{b-a}{Z_b - Z_a} \quad \Rightarrow \quad c_{\text{zéro}} \approx n_{\text{itér}} \cdot t_{\text{appel}}, \quad n_{\text{itér}} \approx 27.$$

Il n'existe **aucun** parallélisme à l'intérieur d'un zéro.

Niveau 3 — coût du run complet. Les n zéros sont **indépendants** → parallélisables sur W workers :

$$T_{\text{total}} \approx \frac{n \cdot n_{\text{itér}} \cdot t_{\text{appel}}}{W}$$

17.2 Exemple chiffré à $t \approx 9000$

Grandeur	Calcul	Valeur
Termes N	$\lfloor \sqrt{9000/2\pi} \rfloor = \lfloor 37,85 \rfloor$	37
Coût d'un appel siegelz	mesuré (dps ≈ 30)	≈ 296 ms
Itérations / zéro	findroot Illinois	≈ 27
Coût d'un zéro $c_{\text{zéro}}$	27×296 ms	$\approx 8,0$ s
Débit (4 workers)	4/8,0	$\approx 0,5$ z/s
Run $T = 10000$	$\approx 10\,142/0,5$ (queue plus lente)	$\approx 5\text{--}7$ h

La latence d'un appel siegelz à grand t est le **goulot mesuré** (§6.5.5) : à $t \approx 9000$, ~ 296 ms pour $N \approx 37$ termes. Réduire $n_{\text{itér}}$ (Newton) ne sert à rien — c'est t_{appel} qu'il faut attaquer.

17.3 Facteur d'accélération requis et leviers

Pour passer de ~ 7 h à la cible de 30 min :

$$\text{facteur} = \frac{T_{\text{actuel}}}{T_{\text{cible}}} \approx \frac{7 \text{ h}}{0,5 \text{ h}} = 14.$$

Position de chaque levier dans $T_{\text{total}} = \frac{n n_{\text{itér}} t_{\text{appel}}}{W}$:

Levier	Agit sur	Gain	Réaliste (i7,
Cœurs W	dénominateur (linéaire)	$\times 14 \Rightarrow W \approx 56$	×
GPU GTX 960M	détection seule (10-20 %)	nul sur l'affinage	×
RAM / swap	hors formule (compute-bound)	nul (swap $\approx \times 1000$ plus lent)	×
$n_{\text{itér}}$ (Newton)	numérateur	réfuté §6.5.5	×
dps	$t_{\text{appel}} \propto \text{dps}^2$	$\approx \times 1,4$ (plancher §17.4)	(!) marginal
Librairie Arb	$t_{\text{appel}} \div 10\text{--}20$	$\approx \times 10\text{--}20$	✓ → $\sim 20\text{--}25$

Nombre de cœurs requis pour la cible (librairie mpmath inchangée) :

$$W_{\text{cible}} = W_0 \cdot \frac{T_0}{T_{\text{cible}}} \approx 4 \times 14 = 56 \text{ cœurs.}$$

Le vrai levier est logiciel, pas matériel : seule la bascule `mpmath.siegelz` → `Arb (acb_dirichlet_hardy_z, arithmétique de boules en C)` réduit t_{appel} d'un ordre de grandeur à précision égale.

17.4 Plancher de précision (rappel §6.5.3)

La précision **absolue** requise à hauteur γ impose un nombre minimal de chiffres significatifs :

$$\#\text{chiffres} \gtrsim \log_{10}(\gamma) + 10 \implies \approx 14 \text{ à } \gamma \approx 9999.$$

On ne peut donc pas réduire dps sous ~ 25 sans perdre le critère LMFDB ($< 10^{-10}$). Tout abaissement se valide sur un échantillon $t \approx 9900$, **jamais** sur $T=1000$.

17.5 Deux régimes, deux livrables

Le mur de latence ne pèse que dans **un** des deux objectifs (rappel §11.1) :

Objectif	Méthode
Vérifier HR jusqu'à T (comptage $n_{\text{calc}} = N(T)$)	Illinois_C pur, sans polish <code>siegelz</code>
Catalogue de positions $< 10^{-10}$	polish <code>siegelz</code> , ou réutiliser le CSV v2 existant

Pour la simple **vérification de HR**, le niveau 2 lent disparaît : le coût d'un appel `siegelz` (§17.1) ne pèse que dans le régime « catalogue de positions ». Ce catalogue existe déjà (CSV v2, 10 142 zéros, 50 dps).

Auteur : hprzeta · Dernière mise à jour : 2 juin 2026 — ~929 lignes

§18 — Profil de pipeline v4.1 : résultats mesurés et leçon workprec (3 juin 2026)

18.1 Profil mesuré (T=1000, 4 workers, branche Riemann_Lab_C)

Phase	temps cumulé (4 workers)	appels	ms/appel	% mur×W
<code>mpmath_petit_t</code>	11.27 s	138	81.65	35.6 %
<code>illinois_C</code>	5.22 s	511	10.21	16.5 %
<code>turing</code>	2.33 s	1	2 330.99	7.4 %
<code>detection</code>	0.09 s	4	21.53	0.3 %

Vitesse globale : 82 z/s (×80 vs v3 à 1.02 z/s). Turing COMPLET, LMFDB 19/20.

18.2 Interprétation du profil

Le noyau Illinois_C est **réellement rapide** en pipeline (10 ms/appel). Le goulot résiduel est `mpmath_petit_t` : les 138 zéros à $t < 300$ affinés par `mpmath.siegelz` à `dps=35`. Ce coût croît en $\mathcal{O}(\sqrt{t})$ par le nombre de termes RS :

$$N(t) = \left\lfloor \sqrt{\frac{t}{2\pi}} \right\rfloor \quad \text{termes dans la somme principale}$$

À $t < 300$, $N < 7$ — la somme est courte, mais l'arithmétique multi-précision (`dps=35` ≈ 116 bits) coûte cher par terme. La détection (`z_vect_correct`) est négligeable.

18.3 Leçon `workprec` — piège `mpmath`

Tentative : `with _mp.workprec(50): _mp.findroot(_mp.siegelz, (a,b), ...)`.

Résultat : gain marginal ($\sim 12\%$). Cause : `mpmath.siegelz` re-lit `mp.dps` **global** (35) à chaque appel interne. Le contexte `workprec` contrôle la précision de la boucle d'itération Illinois, pas la précision des évaluations de la fonction.

`workprec(50)` $\nRightarrow$ `siegelz` à 50 bits

La seule façon d'appeler `siegelz` en float64 natif :

`_mp.fp.siegelz(t)` # float64 pur — précision $\sim 1e-15$, coût $\sim \times 40$ plus faible

18.4 Prochain levier : `fp.siegelz` pour $t < 300$

Pour $t < 300$, $N < 7$ termes et `tol` = $1e-12$ $\rightarrow$ `fp.siegelz` (float64) est amplement suffisant. La précision float64 ($\varepsilon \approx 10^{-16}$) est bien sous la tolérance, et l'erreur de bracket est dominée par `STEP` = 0.1.

Gain estimé : 81 ms $\rightarrow$ ~ 2 ms par appel $\rightarrow$ 138 zéros en ~ 0.3 s au lieu de 11.3 s $\rightarrow$ pipeline T=1000 : ~ 6 s $\rightarrow$ $\sim 200+$ z/s**.

Auteur : *hprzeta* — *Riemann_Lab* — Mise à jour : 3 juin 2026